assignment2_part2

October 22, 2026

1 Assignment 2: Mining Itemsets (Part II)

1.1 Finding Frequent Itemsets with Apriori

In Part I of this assignment, the summary statistics gave us a brief view of what the data look like. Now it is time for the real business - let's use the *Apriori* algorithm to find the frequent itemsets. First, let's import the packages and dependencies that will be used in this part of the assignment.

[1]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MultiLabelBinarizer

from mlxtend.frequent_patterns import apriori

import time
from sklearn.preprocessing import MultiLabelBinarizer

from mlxtend.frequent_patterns import association_rules
```

[ ]:
```
# Ignore warnings generated by mlxtend
```

NOTE: These are all the imports we need to make for this assignment. You should not make other imports in your submitted notebook. You will receive 0 points for the exercises if your solution includes additional imports.

Before you practice Apriori on the Instacart dataset, let's use another dataset as an example to get familiar with the algorithm. For this assignment, **we sampled 10,000 Tweets with two or more food/drink emojis** (yes, those colorful tasty ideograms). You will represent this dataset as a collection of itemsets and practice what we learned in class.

[2]:
```
# Load the tweets data from files.
tweets_df = pd.read_csv("assets/food_drink_emoji_tweets.txt",␣
 ↪sep="\t",header=None)
tweets_df.columns = ['text']

# Only the emojis below are considered in this assignment.
emoji_list = "                                                    "
emoji_set = set(emoji_list)
```

```
# Get a list of emojis used in each tweet.
tweets_df['emojis'] = tweets_df.text.apply(lambda text:np.unique([chr for chr␣
 ↪in text if chr in emoji_set]))

# Create a matrix for Apriori algorithms.
mlb = MultiLabelBinarizer()
emoji_data = mlb.fit_transform(tweets_df.emojis).astype(bool)
emoji_matrix = pd.DataFrame(data=emoji_data, index=tweets_df.index, columns=mlb.
 ↪classes_)
```

We can call the Apriori API now and specify the minimal support we want. You may learn more about this API from its documentation.

[3]:
```
# Apply the Apriori algorithms and find the most frequent items.
freq_emoji_itemsets = apriori(emoji_matrix, min_support=0.005,␣
 ↪use_colnames=True)
freq_emoji_itemsets.sort_values("support", ascending=False).head()
```

[3]:

	support	itemsets
55	0.181663	( )
59	0.148725	( )
22	0.138783	( )
57	0.108255	( )
60	0.103234	( )

With the above command, we preserve all itemsets with a min support of 0.005 (half percent of the shopping baskets). We can now use the following command to extract the frequent itemsets with length 2 and beyond.

[4]:
```
# Apply the Apriori algorithms and find the most frequent 2-itemsets.
freq_emoji_itemsets[freq_emoji_itemsets["itemsets"].apply(
    lambda x: len(x) > 1)].sort_values("support", ascending=False).head()
```

[4]:

	support	itemsets
136	0.081944	(,)
202	0.057040	(,)
169	0.045993	(,)
192	0.042478	(,)
92	0.036654	(,)

Now, it is time to apply the Apriori algorithm to the Instacart dataset.

Since we have already shown you how to transform the Instacart dataset into itemsets, we concatenate the data preprocessing code into one block. Please run the following code block to load and preprocess the data. Notice that we limit the number of products in this problem to 100 by popularity so that the apriori algorithm will not run for a long time.

[5]:
```
# Load data from files.
orders = pd.read_csv("assets/orders.csv.zip", nrows=10000)
products = pd.read_csv("assets/products.csv.zip")

# Group orders by order id and merge them into a list.
order_baskets = orders.groupby("order_id")["product_id"].apply(list)

# Convert the above pandas Series to a pandas DataFrame.
order_baskets = order_baskets.to_frame(name="products_id")

# Create the name map for later reference.
product_name_map = dict(zip(products["product_id"], products["product_name"]))

# Create the matrix like Part 1.
mlb = MultiLabelBinarizer()
data = mlb.fit_transform(order_baskets["products_id"]).astype(bool)
prod_matrix = pd.DataFrame(
    data=data, index=order_baskets.index, columns=mlb.classes_
)
prod_popularity = prod_matrix.sum(axis=0)
prod_matrix = prod_matrix.astype(bool)
top_prods = prod_popularity.sort_values(ascending=False).head(100).index
prod_matrix = prod_matrix[top_prods]
prod_matrix.head()
```

[5]:

order_id	24852	13176	21137	21903	47209	47766	16797	47626	22935	…	27966	41220	5450	5025	16185	39928	23375	7781	21709	…	27344	31915
2	False	False	False	False	False	False	False	False	False	…	False	False	False	False	False	False	False	False	False	…	False	False
3	False	False	False	True	False	False	False	False	False	…	False	False	False	False	False	False	False	False	False	…	False	False
4	False	False	False	False	False	False	False	False	False	…	False	False	False	False	False	False	False	False	False	…	False	False
5	False	True	False	False	True	False	False	False	False	…	True	False	False	False	False	False	False	False	False	…	False	False
6	False	False	False	False	False	False	False	False	False	…	False	False	False	False	False	False	False	False	False	…		

```
6         False  False

[5 rows x 100 columns]
```

[6]:
```
prod_matrix.shape
```

[6]:
```
(977, 100)
```

1.1.1 Exercise 2. (20 pts)

Complete the following `prod_frequent_itemsets` function to find all the **frequent *k*-itemsets** with a minimal support of **`min_support`** in the Instacart dataset.

Your function should return a Pandas DataFrame object like the `freq_emoji_itemsets` object above, being the default format returned by the `apriori` function.

Make sure that you are only returning the frequent itemsets that have the specified number of products (`k`).

Hint:

Code in the previous blocks shows you how to obtain itemsets with a length of greater than 1. How might you change the section of the code `freq_emoji_itemsets[freq_emoji_itemsets["itemsets"].apply(lambda x: len(x) > 1)]` to instead obtain itemsets equal to (==) k instead of greater than (>) 1?

[11]:
```
def prod_frequent_itemsets(prod_matrix, min_support=0.005, k=3):
    freq_sets = ""
    # YOUR CODE HERE
    #raise NotImplementedError()
    freq_sets = apriori(prod_matrix, min_support=min_support, use_colnames=True)
    freq_sets = freq_sets[freq_sets["itemsets"].apply(lambda x: len(x) == k)].
 ↪sort_values("support", ascending=False)

    return freq_sets
```

If you implemented this function correctly, we can obtain all frequent 2-itemsets with a min support of 0.004 by running the following command.

[12]:
```
prod_frequent_3itemsets = prod_frequent_itemsets(prod_matrix, min_support=0.
 ↪004, k=3)
prod_frequent_3itemsets
```

[12]:

	support	itemsets
381	0.007165	(13176, 21137, 47209)
376	0.005118	(45007, 24852, 21903)
378	0.005118	(25890, 24852, 21903)
374	0.004094	(21137, 24852, 21903)
375	0.004094	(24852, 47766, 21903)

```
377  0.004094  (19057, 24852, 21903)
379  0.004094  (45066, 24852, 16797)
380  0.004094  (45066, 24852, 27845)
382  0.004094  (13176, 47209, 27966)
383  0.004094  (43352, 12341, 16797)
```

[13]:
```
prod_frequent_3itemsets = prod_frequent_itemsets(prod_matrix,
                                                 min_support=0.004,
                                                 k=3)
for row in prod_frequent_3itemsets.itertuples():
    assert row.support >= 0.004, f"[Exercise 2] The support of the itemset {row.
 ↪itemsets} is below the threshold."
    assert len(row.itemsets) == 3, f"[Exercise 2] The itemset {row.itemsets} is␣
 ↪not a 3-itemset."
```

Let's replace the product IDs with their names. Does the result make sense to you?

[14]:
```
prod_frequent_3itemsets.itemsets = prod_frequent_3itemsets.itemsets.
 ↪apply(lambda row: [product_name_map[i] for i in row])
prod_frequent_3itemsets
```

[14]:
```
      support                                            itemsets
381  0.007165  [Bag of Organic Bananas, Organic Strawberries,…
376  0.005118   [Organic Zucchini, Banana, Organic Baby Spinach]
378  0.005118  [Boneless Skinless Chicken Breasts, Banana, Or…
374  0.004094  [Organic Strawberries, Banana, Organic Baby Sp…
375  0.004094    [Banana, Organic Avocado, Organic Baby Spinach]
377  0.004094  [Organic Large Extra Fancy Fuji Apple, Banana,…
379  0.004094           [Honeycrisp Apple, Banana, Strawberries]
380  0.004094     [Honeycrisp Apple, Banana, Organic Whole Milk]
382  0.004094  [Bag of Organic Bananas, Organic Hass Avocado,…
383  0.004094         [Raspberries, Hass Avocados, Strawberries]
```

1.2 Evaluating Frequent Itemsets (Part III.)

Even though we have found all the frequent itemsets, not all of them are interesting. In Part IV of this assignment, we will practice how to evaluate the frequent itemsets.

People have developed various measurements of the interestingness of patterns. Most of them split the itemset into an antecedent item(set) and a consequent item(set), and then measure the correlation between the antecedent and the consequent. Let's try some of such measurements implemented by the `mlxtend.frequent_patterns.association_rules` API, which we have imported. For more information about the API, visit the documentation of the `mlxtend` package.

[15]:
```
prod_frequent_itemsets_basic = apriori(prod_matrix, min_support=0.005,␣
 ↪use_colnames=True)
```

```
interestingness_measurements = association_rules(prod_frequent_itemsets_basic,␣
 ↪metric="lift", min_threshold=0)
interestingness_measurements.head(10)
```

[15]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction	zhangs_metric
0	(21137)	(24852)	0.074719	0.159672	0.012282	0.164384	1.029505	0.000352	1.005638	0.030973
1	(24852)	(21137)	0.159672	0.074719	0.012282	0.076923	1.029505	0.000352	1.002388	0.034105
2	(24852)	(21903)	0.159672	0.071648	0.024565	0.153846	2.147253	0.013125	1.097143	0.635810
3	(21903)	(24852)	0.071648	0.159672	0.024565	0.342857	2.147253	0.013125	1.278759	0.575524
4	(47209)	(24852)	0.069601	0.159672	0.008188	0.117647	0.736802	-0.002925	0.952371	-0.277424
5	(24852)	(47209)	0.159672	0.069601	0.008188	0.051282	0.736802	-0.002925	0.980691	-0.298291
6	(24852)	(47766)	0.159672	0.054248	0.016377	0.102564	1.890663	0.007715	1.053838	0.560597
7	(47766)	(24852)	0.054248	0.159672	0.016377	0.301887	1.890663	0.007715	1.203712	0.498106
8	(24852)	(16797)	0.159672	0.053224	0.012282	0.076923	1.445266	0.003784	1.025674	0.366626
9	(16797)	(24852)	0.053224	0.159672	0.012282	0.230769	1.445266	0.003784	1.092426	0.325405

In the returned data frame, each row examines one (antecedent -> consequent) pair. *Antecedent support* and *consequent support* measure P(antecedent) and P(consequent), while *support* measures P(antecedent, consequent). In fact, these three values help us characterize the 2×2 contingency table, as illustrated in the following table:

	X = 1	X = 0	sum(row)
Y = 1	`support`		`cons_support`
Y = 0			
sum(col.)	`ante_support`		

Most interestingness measurements, including the four shown in the data frame (*confidence*, *lift*, *leverage*, and *conviction*), can be derived from the three support values. For example,

$$\text{confidence} = \frac{\text{support}}{\text{antecedent_support}},$$

and

$$\text{lift} = \frac{\text{confidence}}{\text{consequent_support}} = \frac{\text{support}}{\text{antecedent_support} * \text{consequent_support}}$$

1.2.1 Exercise 3. (15 pts)

In this exercise, we are going to implement another interestingness measurement, the (full) mutual information which we will use when adding a 'mutual information' column to the data frame. The measurement is defined as

$$I(X;Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} P(X = x, Y = y) \log_2 \frac{P(X = x, Y = y)}{P(X = x)P(Y = y)}.$$

(*The mutual information quantifies the "amount of information obtained from one random variable by observing the other random variable.*)

Note that the logorithm requirest that the joint probability $P(X = x, Y = y) > 0$, which does not hold for some (x, y). However, since we know that when $P(X = x, Y = y) = 0$, it would not contribute to the sum, you may assume $P(X = x, Y = y) \log_2 \frac{P(X=x,Y=y)}{P(X=x)P(P=y)} = 0$ in that case.

x, y are possible values of X and Y; in the case of appearance or absence of an item, 1 or 0. Therefore, we need to consider all possible combinations of x and y, that is, $(X = 1, Y = 1)$, $(X = 1, Y = 0)$, $(X = 0, Y = 1)$, $(X = 0, Y = 0)$.

Please complete the following function that uses the three support values to compute the mutual information and return it as a float. All the three parameters are in [0, 1], and you can assume the validity of the input. **Use 2 as the log base.** We have created some auxilary variables for you, each represent a joint or marginal (let X denote the antecedent item and Y denote the consequent item) probability.

[16]:
```
def mi(antecedent_support, consequent_support, support):

    px1 = antecedent_support
    px0 = 1 - antecedent_support
    py1 = consequent_support
    py0 = 1 - consequent_support

    px1y1 = support
    px1y0 = px1 - px1y1
    px0y1 = py1 - px1y1
    px0y0 = 1 - px1 - py1 + px1y1

    mutual_information = 0

    # YOUR CODE HERE
    # raise NotImplementedError()
    if px0y0 > 0 and px0 > 0 and py0 > 0:
        mutual_information += px0y0 * (np.log2(px0y0) -  np.log2(px0) - np.
↪log2(py0))
    if px0y1 > 0 and px0 > 0 and py1 > 0:
        mutual_information += px0y1 * (np.log2(px0y1) -  np.log2(px0) - np.
↪log2(py1))
    if px1y0 > 0 and px1 > 0 and py0 > 0:
```

```
        mutual_information += px1y0 * (np.log2(px1y0) -  np.log2(px1) - np.
↪log2(py0))
    if px1y1 > 0 and px1 > 0 and py1 > 0:
        mutual_information += px1y1 * (np.log2(px1y1) -  np.log2(px1) - np.
↪log2(py1))

    return mutual_information
```

[17]:
```
# This code block tests whether the `mi` function work as expected.
# We hide some tests, so passing all the displayed assertions does not␣
↪guarantee the bonus points.

assert np.abs(mi(0.6, 0.75, 0.4) - 0.04287484674660057) < 1e-8
assert np.abs(mi(0.5, 0.5, 0.25) - 0) < 1e-8

# If you fail the following assertion, double check if your function
# handles the scenarios in which a joint probability is zero.
assert np.abs(mi(0.5, 0.5, 0.5) - 1) < 1e-8
```

With the `mi` function, we can now compute the mutual information for each (antecedent -> consequent) pair and attach it to the data frame. Does the result make sense?

Mutual information is a classical measure of interestingness. We encourage you to think about the following questions (not graded): 1. What is the maximum of mutual information in this setting? How to reach it? 2. What is the minimum of mutual information in this setting? How to reach it?

What does the max/min value imply?

[18]:
```
interestingness_measurements['mi'] = \
    interestingness_measurements.apply(lambda pair: mi(pair['antecedent␣
↪support'],
                                                       pair['consequent support'],
                                                       pair['support']),
                                       axis=1)
interestingness_measurements.sort_values('mi', ascending=False).head(n=5)
```

[18]:

	antecedents	consequents	antecedent support	consequent support	support	\
260	(43352)	(16797)	0.026612	0.053224	0.010235	
261	(16797)	(43352)	0.053224	0.026612	0.010235	
93	(47209)	(13176)	0.069601	0.121801	0.023541	
92	(13176)	(47209)	0.121801	0.069601	0.023541	
282	(24964)	(45007)	0.035824	0.029683	0.008188	

	confidence	lift	leverage	conviction	zhangs_metric	mi
260	0.384615	7.226331	0.008819	1.538511	0.885174	0.020237
261	0.192308	7.226331	0.008819	1.205147	0.910054	0.020237
93	0.338235	2.776940	0.015064	1.327056	0.687760	0.017598
92	0.193277	2.776940	0.015064	1.153307	0.728641	0.017598

```
282    0.228571  7.700493  0.007125    1.257819       0.902468  0.016409
```

[ ]: